Quantum-Assisted Hypothesis Search

A plain-language guide to what the current code does, where QAOA fits, what has already been verified, and how to test the research claim.

The one-sentence version

The program faces a hidden-answer problem. It lists possible explanations and possible actions, scores which action could teach it the most, then lets either a classical search method or QAOA choose a hypothesis-action candidate. After the environment responds, the program removes explanations that no longer fit and repeats.

> **The researchers know the hidden answer so that the run can be graded. The agent does not receive that answer. QAOA is useful only if it can guide the agent through the search more effectively than a matched classical method.**

The full loop

1. **Observe:** The harness receives the visible state and the history of earlier actions.
2. **Propose:** An interpreter supplies candidate explanations. The current default is rule-based; an OpenAI adapter exists but is not wired into the runners.
3. **Predict:** For every explanation, the code predicts what each available action should cause.
4. **Score:** Actions receive higher scores when they could reduce uncertainty or make progress toward the goal.
5. **Encode:** The candidate explanation and action choices are translated into bits. These bits are the decision variables used by classical search or QAOA.
6. **Search:** The selected backend searches for a low-cost bitstring. Lower cost means better according to the scoring rules.
7. **Act and learn:** The environment executes the selected action, returns an observation, and the program updates its belief in each explanation.
8. **Repeat:** The new evidence changes the scores, so the next search solves a slightly different problem.

What QAOA contributes

QAOA does not invent the possible explanations, understand the game, or decide what counts as useful evidence. Those parts are supplied classically. QAOA receives a cost function and tries to put more probability on low-cost bitstrings. The research question is whether this search step leads to the hidden answer with fewer actions or better reliability under the same budget.

1. The small red-tile teaching example

This is the clearest place to understand the program. A blue player sees a red tile and a green door. The red tile has one hidden role.

Possible explanation	What touching red would do
Red is a hazard	Destroy the player
Red is a switch	Open the green door
Red is the goal	Finish the game
Red is irrelevant	Cause no visible change

> **There are four hypotheses and two actions. Eight hypothesis-action combinations fit into three decision bits because 2^3 = 8.**

What the main files do

File	Plain-language job
environment.py	Hides the true red-tile rule and returns the result of an action.
interpreter.py	Creates the four candidate explanations. It also defines the API-ready language-model interface.
predictor.py	Says what each hypothesis predicts for each action.
objective.py	Calculates information gain, progress, costs, and updated beliefs.
compiler.py	Maps a hypothesis and action to a three-bit index and decodes it later.
agent.py	Runs the observe -> score -> search -> act -> update loop.

Example: the hidden rule is 'red is a switch'

1. **Start uncertain:** All four explanations begin at probability 0.25.
2. **Score actions:** Approaching red is valuable because its four possible outcomes separate all four explanations. Approaching green distinguishes much less.
3. **Select and act:** The search backend chooses approach_red.
4. **Receive evidence:** The environment reports that the green door opened.
5. **Update beliefs:** The switch explanation rises above 99.9 percent; contradictory explanations receive almost no belief.
6. **Finish:** The next scores favor approach_green, which reaches the goal.

This example validates the loop. It is too small and too obvious to demonstrate a quantum advantage.

2. What the three search backends actually do

Backend	What it does	What it proves
Greedy baseline	Picks the action with the largest immediate utility, then pairs it with the best-believed hypothesis.	A simple policy can use the same scores.
Exact classical	Calculates every valid candidate cost and takes the minimum.	The scoring and encoding produce the expected optimum on small problems.
QAOA statevector	Tunes a simulated quantum circuit so low-cost bitstrings become more likely, samples it, then keeps the best valid sample.	The QAOA plumbing can recover useful candidates on small instances.

Inside the current QAOA backend

1. **Uniform start:** Every bitstring starts with equal amplitude. No answer is preferred yet.
2. **Cost phase:** The known scoring function changes the phase of each bitstring according to its cost.
3. **Mixer:** Rotation gates move amplitude among bitstrings.
4. **Classical tuning:** COBYLA changes the beta and gamma angles to reduce the circuit's expected cost. The current depth is normally p = 2.
5. **Measurement simulation:** The code samples bitstrings according to the final statevector probabilities.
6. **Valid result:** It rejects invalid encodings and chooses the lowest-cost sampled candidate.

Important distinction

We know how to score a proposed answer; we do not know which proposal will score best until search is performed. QAOA can only optimize relationships represented in the cost. It cannot discover a rule that the hypotheses, predictions, and objective never expressed.

Current simulation shortcut

The implementation creates the complete cost list and complete quantum state in ordinary NumPy memory. That is appropriate for debugging, but it grows as 2^qubits. It therefore demonstrates the algorithm's logic, not scalable quantum computing.

Current object	Meaning
costs	One classical number for every possible bitstring
statevector	One complex amplitude for every possible bitstring
beta, gamma	Circuit settings tuned anew for the current cost problem
shots	Repeated samples from the final probability distribution

3. The larger 64-world experiment

The second path replaces the four red-tile explanations with six binary rule dimensions. Choosing option 0 or 1 for every dimension creates 2^6 = 64 complete world models.

Rule dimension	Two possible values
Player	Blue is controlled / nearest object is controlled
Goal	Reach green / reach red
Movement	Move one cell / slide until an obstacle
Topology	Gray is a wall / gray contains a passable gap
Interaction	Red is a hazard / red is a switch
State	Stateless / sequence dependent

Encoding

Six bits identify one of the 64 hypotheses. Seven additional one-hot bits represent six probe actions and one solution attempt. Exactly one action bit should be 1. The complete QAOA problem therefore uses 13 decision qubits and a simulated statevector with 2^13 = 8,192 amplitudes.

What QUBO means here

A Quadratic Unconstrained Binary Optimization model is a score written using binary variables, individual-bit terms, and bit-pair terms. The code adds a large penalty if zero or several action bits are selected. It adds consistency penalties for rule values already observed and rewards informative actions.

How a run behaves

Each probe directly reveals one of the six hidden bits. Because all six begin equally uncertain, the agent normally performs six probes and then attempts the complete solution on action seven. This is why the policy proxy and exact classical search both solve all 64 worlds in seven actions.

The main scientific weakness

The current QUBO mostly chooses the best hypothesis bits and the best action independently. It contains penalties between action bits, but it does not yet contain rich terms connecting a particular hypothesis pattern to a particular action's outcome. The environment also reveals one rule bit per probe. This makes the experiment excellent for checking software and weak for showing that QAOA found a difficult reasoning path.

What a stronger next environment needs

Actions should produce partial or ambiguous observations that support several hypotheses at once. Some actions should be locally attractive but poor over multiple steps. The hidden answer must remain available to the evaluator while being absent from the agent's inputs.

4. How the codebase fits together

Layer	Files	Responsibility
Teaching environment	models.py, environment.py	Defines the small hidden-rule game and shared records.
Candidate generation	interpreter.py, providers/openai_provider.py	Creates hypotheses deterministically or through an API-ready language-model call.
Prediction and scoring	predictor.py, objective.py	Predicts outcomes, calculates information gain and progress, and updates beliefs.
Small encoding	compiler.py	Encodes four hypotheses and two actions into three bits.
Search	search/greedy.py, classical.py, qaoa.py	Provides three interchangeable methods for selecting a candidate.
Loop and storage	agent.py, run_store.py, cli.py	Runs episodes, saves traces, and provides the command-line demo.
64-world representation	factorized.py, rule_environment.py	Defines six binary rule dimensions, seven actions, and the controlled hidden world.
64-world objective	factorized_objective.py, qubo.py	Scores probes, builds the 13-variable QUBO, and converts it to Ising form.
Experiments	factorized_agent.py, benchmark.py, benchmark_cli.py	Runs matched conditions and records aggregate metrics.
Future integration	environment_adapter.py	Defines the minimal reset/step interface for ARC or another environment.
Interfaces	dashboard/server.py, dashboard/app.py	Displays and saves runs from the small teaching example.

What is connected today

The command line and dashboards run the small red-tile example. The benchmark command runs the 64-world experiment. The OpenAI caller and generic environment adapter are available building blocks, but neither is connected to the main experimental path. The current quantum backend is a custom NumPy statevector simulator; Qiskit is not yet a dependency.

What was verified during this review

All nine unit tests passed. The classical and QAOA small demos both solved the switch world in two turns: approach red, observe the door open, then approach green. Across all four hidden rules, all three backends succeeded on the switch and goal worlds but failed on the hazard and does-nothing worlds. The information-seeking action is fatal in the hazard world; after learning that red does nothing, the loop can repeat zero-value actions. These are objective and stopping-rule limitations shared by every backend.

The preserved 64-world results show both classical conditions at 64/64 with seven actions on average. The preserved eight-world QAOA smoke run solved 8/8 with 6.875 actions on average, but one early correct guess and post-development tuning prevent an advantage claim.

5. Checklist A - test what already exists

Run these from the QAHS-Prototype folder.

Software correctness

[ ] Run: `PYTHONPATH=src python3 -m unittest discover -s tests -v`

[ ] Confirm all nine tests pass.

[ ] Run the classical and QAOA small demos with the hidden rule `red_is_switch`.

[ ] Confirm both choose red first, observe an open door, and then choose green.

[ ] Repeat the small demo for all four hidden rules and record success, turns, and actions.

[ ] Reproduce the current limitation: every backend fails the hazard and does-nothing worlds. This is a shared objective/stopping issue, not a QAOA-specific failure.

Visual inspection

[ ] Start: `PYTHONPATH=src python3 dashboard/server.py`

[ ] Open `http://127.0.0.1:8000`.

[ ] Run the same hidden rule with Greedy, Classical exact, and QAOA.

[ ] Confirm action scores are identical before search; only the search method should change.

[ ] Confirm the belief update agrees with the returned observation.

[ ] Confirm a JSON trace appears under `output/runs/`.

Controlled benchmark

[ ] Run the policy proxy and exact classical search across all 64 worlds.

[ ] Confirm 64/64 success and about seven actions per world.

[ ] Run QAOA on eight worlds first; record the seed, depth, restarts, optimizer limit, shots, time, and actions.

[ ] Treat this as a smoke test. Eight worlds cannot establish an advantage.

Recommended commands

Purpose	Command
64 classical worlds	PYTHONPATH=src python3 -m qahs.benchmark_cli --conditions policy_proxy classical --worlds 64 --output output/experiments/reproduction_classical_64.json
8 QAOA worlds	PYTHONPATH=src python3 -m qahs.benchmark_cli --conditions qaoa --worlds 8 --output output/experiments/reproduction_qaoa_8.json

6. Checklist B - turn it into a fair experiment

Freeze the comparison

[ ] State the primary claim before final testing. Example: QAOA-assisted search reduces actions to solution under an equal search budget.

[ ] Give every condition the same state, history, hypotheses, actions, scoring rule, and stopping rule.

[ ] Compare a real LLM-only action choice, greedy classical, exact classical, and QAOA-assisted search.

[ ] Freeze QAOA depth, restarts, optimizer limit, shots, penalties, and seeds.

[ ] Separate development tasks from held-out test tasks.

[ ] Use solve rate and actions to solution as primary metrics.

[ ] Also report time, model calls, shots, optimizer evaluations, qubits, and circuit depth.

[ ] Repeat stochastic conditions across multiple seeds and report uncertainty.

Connect the language model

[ ] Install the optional LLM dependency and set the API key only in the environment.

[ ] Wire OpenAIJSONCaller into a runner; it is not currently activated by the command line or dashboard.

[ ] Freeze the prompt and output schema before held-out tests.

[ ] Save every proposed hypothesis, candidate action, model response, and parse failure.

[ ] Add an LLM-only baseline that chooses the next action without QAHS.

Use a meaningful environment

[ ] Implement an ARC or other task through the reset/step adapter.

[ ] Keep hidden truth in the evaluator and out of the agent prompt and scoring inputs.

[ ] Remove direct probes that reveal exactly one hidden bit.

[ ] Use ambiguous outcomes that support several competing explanations.

[ ] Include cases where the best immediate action is not the best multi-step plan.

[ ] Test enough unseen tasks to distinguish conditions.

7. Checklist C - validate the quantum part

Formulation checks

[ ] Confirm every valid bitstring decodes to exactly one hypothesis and one action.

[ ] Confirm invalid encodings cost more than any possible reward.

[ ] Check QUBO and Ising energies on many random bitstrings, not only the four unit-test examples.

[ ] Add meaningful hypothesis-action interaction terms when the environment supports them.

[ ] On small problems, compare the exact minimum with the best candidate QAOA sampled.

[ ] Record the probability placed on optimal and near-optimal candidates.

[ ] Increase qubits gradually and record simulator memory, optimizer work, and failure rate.

Qiskit checks

[ ] Build the Ising cost operator in Qiskit.

[ ] Run Qiskit QAOA on a noiseless simulator and compare it with the current NumPy result.

[ ] Transpile and report logical qubits, circuit depth, one-qubit gates, and two-qubit gates.

[ ] Run a noise model and report how often the best answer is recovered.

[ ] If hardware is available, use a small frozen instance and record calibration, shots, and errors.

Interpretation checks

[ ] Describe working demos as software validation.

[ ] Claim better search only when held-out matched comparisons improve the primary metrics.

[ ] Report where QAOA is slower, less stable, or no better.

[ ] Keep 'quantum-assisted reasoning' separate from 'quantum speedup.'

Current readiness

Question	Current answer
Does the full observe-search-act-update loop run?	Yes, on synthetic teaching environments.
Can classical and QAOA use the same encoded objective?	Yes.
Does the current code use a real LLM?	No. An adapter exists but is not connected.
Does it run genuine ARC tasks?	No. The adapter is only a contract.
Does it use Qiskit or quantum hardware?	No. It uses a NumPy statevector simulator.
Does it establish stronger AI or quantum advantage?	No. That requires the held-out matched study above.

The next useful milestone

Connect one real model, one non-trivial interactive task, and all four matched conditions. Freeze the prompt and QAOA settings, then run a small development study. That result will tell you whether to improve the objective, the environment, or the quantum search before scaling the experiment.